

Reproducibility in the Age of Agentic AI: Context Engineering at the Timescale of a Codebase

Lorena A. Barba

The George Washington University, Washington, D.C.

June 2026

Abstract

Reproducible research practices are context engineering for AI coding agents. I argue that agents lower the cost of maintaining tests, commit histories, repository structure, instructions, and decision records while making their benefits immediate. Researchers remain responsible for verifying these artifacts and the scientific judgments they encode.

Since the start of my career as a principal investigator, I have been advocating for reproducible research practices. It was the upshot of the toils of my own PhD studies. [1] Version control, open code development and public release, sharing of research artifacts like figures and data, and posting preprints were all part of my Reproducibility PI Manifesto more than a decade ago. [2] Over the years, my group established our internal habits for code testing, maintaining clean repository layouts, writing commit messages that explain *why* and not just *what*: these are practices I encouraged on every student cohort, in every course, in every collaboration. Despite the copious writing and speaking about reproducibility, by myself and many others, the uptake has been modest. Friction remains a powerful argument. Each of these practices costs time and effort on a per-session basis that pays back only over many sessions, and graduate students live under the kind of immediate-deadline pressure that punishes long-horizon investment. The research community has accepted it as the cost of doing science with people who are not trained or paid to be software engineers. I admit to becoming disenchanted over the years, even a bit cynical. It thus came as a revelation to find that the calculus has changed, and to see how generative AI is spurring new interest in reproducibility. In this paper, I put forward an updated view on computational reproducibility, for the time of agent-driven code development.

1 Context Is Everything for AI Agents

While preparing a class for my recent course on Generative AI for Engineering Research, [3] I was working through the exercise of how to dissect an existing agentic system and explain its parts through the vocabulary of context design. The exercise was meant to be diagnostic: take a working LLM-based automation, name its context-engineering decisions, identify where the design is principled and where it is incidental. What emerged was a bit of a surprise. *Every* significant design choice in the system could be expressed in terms of context. The static files the agent reads at startup are its always-on semantic memory. The structured outputs it produces are its episodic record. The pre-processing applied before content reaches the model is attention-budget

management. The narrowing of the agent's tool surface is scope control. After working through the system components like this, there was nothing left over. A working agentic system is a collection of context-engineering decisions: stacked in layers, made at different times, often by different hands, but reducible to that single discipline.

That observation is interesting, but the next step is more consequential. Once you can read a system as an implementation of context engineering, you can flip the question: how might you *design* a system so that an agent can be maximally productive in it? This is a different question. Inspecting and assessing are diagnostic while designing is generative. And once you start asking the design question seriously, you find that it fits in a new field, opening the door for both serious concerns and clear opportunities.

2 Good Software Practices Are Now Context Design

The next surprise was that the answer to "how do I make this system agent-friendly" was largely aligned to "how do I make this system reproducible." We can now reframe good research software engineering practices in the vocabulary of context engineering, while promoting reproducible science. A test suite, a clean commit history, a well-organized repository with a tidy tree, a README, a conventions file, conventional directories, and finally a decision record: each of these elements maps onto a category of agent context, and each can now be produced in a new agent-driven way, changing the economics. The table below enumerates these known elements, matching them with their newly interpreted context category. These are not simple analogies: the mapping is strict. Tests, commits, structure, and decision records are **context engineering at the timescale of a codebase** rather than the timescale of a conversation. Good software practices have just acquired a new and entirely mechanical reason for existing: agents need them in order to work productively and correctly.

2.1 Catalog of Agent-Facing Artifacts

The claim that good software practices in agent-driven development are context engineering becomes operational after we draw the concrete mapping. Let's consider each artifact that a well-kept research software repository contains, name the context role it plays, and visualize the new and nearly frictionless way it now comes into being. The unifying point is that you no longer manually write these artifacts. In agentic code development your role shifts from author to **owner and verifier**: the agent drafts the test, the message, the record, and you read it, correct it, and take responsibility for it. The artifact is no less yours for having been drafted by a machine, just as a number produced by a solver you wrote is still your result. You do not need to type the code to own the judgments involved.

The five listed artifacts predate LLMs by years or decades, and each was advocated, with limited success, throughout the era when research code was read only by humans. Two things are new: the cost of producing them has collapsed, because an agent drafts them on request; and they have acquired a second reader, the agent itself, which consults them at the start of and throughout every session. As a bonus, peers can more easily (supported by their agents) reproduce and build upon the work starting from openly shared research artifacts. An artifact that used to pay back slowly over the life of a project now also pays back immediately, in the quality of the next interaction. Let's review them individually.

Artifact	Context category	Reproducibility function	Agent-friendliness function	Now produced by
Agent instruction file: AGENTS.md / CLAUDE.md	Always-on semantic memory	Conventions of record: how the project is built and run	Startup context every session; no re-explanation	Agent drafts from the repo; you add the tacit parts
Test suite (characterization tests)	Executable semantic memory	Behavior made verifiable, not asserted	Self-check signal the agent runs on its own work	Agent drafts tests for trusted functions; you confirm intent
Commit history (Conventional Commits)	Episodic memory	Provenance of every change	Recent trajectory reconstructed from <code>git log</code>	Agent drafts the message from the diff; you edit and own it
Repository structure	Pre-processed context	A layout that can be inherited, not abandoned	Minimal exploration; attention spent on the task	Agent scaffolds from a prompt; you adjust
Decision record (DECISIONS.md)	Static context	The reasoning that links intent to implementation	Settled questions are not relitigated	Agent drafts the entry; you supply the reasoning

The agent instruction file (AGENTS.md or CLAUDE.md) is your project’s always-on semantic memory: the document an agent reads at the start of every session, encoding what the project is, how it is built and run, the conventions it follows, and the boundaries it must respect. An analysis of 2,500 such files in public repositories found a sharp divide between the ones that help and the ones that do not: a vague file ("a helpful coding assistant") yields vague behavior, while an effective file specifies six things: the commands the agent may run, how tests are invoked, the project structure, the code style, the git workflow, and explicit boundaries on what must never be touched. [4] The most useful convention to emerge from that analysis is a three-tier boundary, *always do, ask first, never do*, which turns the file from documentation into operative instruction. Note that you do not write it from scratch: the agent reads the repository and drafts it, and you supply the tacit knowledge that is invisible in the code, such as which data files are sacred or why a particular numerical convention is firmly adopted. As a reproducibility artifact it is your conventions of record, a single human- and machine-readable statement of how the project is meant to be used. As an agent artifact it is the context that keeps every session from beginning in ignorance.

The test suite is executable semantic memory: a specification of expected behavior in a form that can be run and checked. For research code the most valuable pattern is the **characterization test**, which captures the current, trusted behavior of a function without trying to prove it theoretically correct.¹ Thirty characterization tests of the thirty functions whose correctness carries your scientific claims are worth more than three hundred tests of trivial accessors. This is where an agent’s help can be crucial, because writing the first test was historically the highest-friction practice of all: you name a function you already trust, the agent drafts tests that pin its behavior, and you check that the behavior it captured is the behavior you intended. As a reproducibility artifact the suite of tests makes your code’s behavior verifiable rather than merely asserted, and turns every later change into a checkable event. As an agent artifact it is the signal the agent runs to verify its own work, closing the loop between generation and verification that unsupervised generation otherwise leaves open.

¹The term originates with Michael Feathers, *Working Effectively with Legacy Code* (2004) [5]. See the review on the Better Scientific Software blog [6].

The commit history is episodic memory: an ordered account of what changed, when, and why. The *Conventional Commits* convention prefixes each message with a type—`feat`, `fix`, `refactor`, `docs`, `test`, `chore`—followed by an imperative summary and, where it matters, a body that explains the reasoning rather than restating the diff. [7] The distance between "update solver" and "fix(solver): correct sign of the Coriolis term in the v-momentum equation," followed by two lines on how the error survived the rotating-frame test and which published case finally exposed it, is the distance between a log that is noise and one that saves a successor two weeks of bisection eighteen months later. The agent removes the tedium that historically defeated this discipline: it reads the staged diff faster than you could describe it and drafts a message you edit and own. As a reproducibility artifact the history is the provenance of every change. As an agent artifact it lets an agent reconstruct the project's recent trajectory in a few hundred tokens, by reading the log rather than re-deriving the state.

The repository structure is pre-processed context: the physical layout, which an agent reads simply by listing the directory tree. A repository ready for both human and machine readers has a predictable skeleton—a README, an instruction file, a license, a changelog, a decision record, and a dependency specification at the root; source under `src/`, tests under `tests/`, runnable examples under `examples/`, documentation under `docs/`. The working test is whether a new lab member, or an agent on its first session, can orient in under a minute by reading only the root. This is the substance of what has been called **AI-friendly codebase design**: the shape of the repository is itself a context interface, and its quality sets how much of every later interaction is spent on the task rather than on exploration. [8] An agent will scaffold the structure from a single prompt; your contribution is to fit it to the realities of the project. As a reproducibility artifact a legible layout is what lets code be inherited rather than abandoned when its author graduates. As an agent artifact it is what keeps the agent's attention budget on the work rather than on finding where things are.

The decision record (`DECISIONS.md`) is static context: a running log of consequential choices and the reasoning behind each, in the format long used for **Architecture Decision Records**—context, decision, alternatives considered, consequences.² It is the rarest of these artifacts in research software repositories and the most valuable for agentic work, because it preserves exactly what neither the code nor the tests can show: the reasoning that connected a goal to the particular implementation chosen to meet it. The example below shows the form:

2026-03-14: Use sparse CSR format throughout the linear solver

Context: Memory became the binding constraint at grids above 512^3 . Dense storage would exceed available RAM.

Decision: Use `scipy.sparse` CSR matrices for all stiffness and mass matrices. Do not convert to dense for debugging prints.

Alternatives considered: COO (simpler, but slower matrix-vector products); dense (rejected for memory reasons); JAX sparse (rejected for dependency complexity).

Consequences: All solver code must tolerate sparse inputs. Visualization utilities that currently assume dense arrays will need `.toarray()` calls, documented in `CLAUDE.md`.

Without such a record, returning to a convention months later—or meeting it for the first time, as an agent does—you cannot tell whether it was deliberate or incidental, and you will either keep it without understanding or discard it and walk straight back into the problem it was chosen to

²Michael Nygard introduced the Architecture Decision Record in a 2011 post; the community-maintained spec is available online [9].

avoid. With the record, the decision becomes essential context that any reader, human or machine, recovers in seconds. As a reproducibility artifact the decision record closes the gap between what was done and why it was done that way. As an agent artifact it stops the agent from reopening questions you have already settled.

2.2 Injecting rigor: the invocation layer

A reasonable objection emerges here. If an agent session runs for hours across dozens of turns, must you document every prompt to preserve a record of how the work was done? In my opinion, that target is both unmanageable and mistaken. The transcript of a long, exploratory session is not the artifact worth keeping; much of it could be dead ends, false starts, and reformulations, the very material you would never transcribe from your own thinking. What is worth keeping is what the catalog already identifies: the decisions, the verified behavior, the rationale, written to project files. **Provenance, in agentic research, lives in the maintained artifacts, not in the conversation that produced them.**

The practical question is how those artifacts stay current without your stopping the work to write them. The answer is a new stratum of the scientific programmer's literacy: controlled invocation. Coding agents now expose **slash commands** and **skills**, named and reusable operations that you trigger deliberately and that write to project files in a constrained, repeatable way. A `/commit` command drafts a Conventional Commits message from the staged diff; a decision-record skill appends a correctly formatted entry from a one-line statement of the choice and its rationale; a changelog skill updates the human-visible history at a release boundary. Each invocation yields a small, well-formed artifact as a byproduct of the work rather than as an interruption to it, and because the operation is named and reusable, it is applied consistently instead of only when you happen to remember.

This is the genuinely new part of the practice. The discipline of reproducible computational work no longer consists only in writing code and managing data with care. It now also consists in knowing how to drive agents so that the record of the work writes itself, correctly and under control, as the work proceeds. Learning the invocation layer—which commands and skills to define, and when to call them—is becoming as much a part of scientific programming as learning the shell once was. The catalog names the artifacts required for reproducibility; the invocation layer is how, in agentic development, you produce them without the friction that used to stand in the way.

3 A Convergence, Not a Coincidence

The mapping in the previous section invites a skeptical reading: that it is a clever relabeling, reproducibility dressed in fashionable new vocabulary. A strong answer to that skepticism comes from people who were not thinking about reproducibility at all.

In the GitHub analysis introduced above, the authors set out to answer a purely instrumental question: across thousands of real repositories, what distinguishes an agent instruction file that makes a coding agent productive from one that does not? Examining more than twenty five hundred files, they found that the effective ones converge on six areas: commands, testing, project structure, code style, git workflow, and boundaries. Nothing in the study was motivated by scientific integrity, transparency, or the replication crisis; the goal was throughput, getting useful

work out of an agent. These six areas are the same practices named with the artifact catalog, now seen from the agent's side of the table.

Read that list again with a reproducibility eye. Commands are the runnable instructions for building and executing the project, the "how to run it" that every reproducible-research checklist demands. Testing is verification, whose absence is the most commonly cited weakness of research software. Project structure is the organized, navigable layout that lets code be audited and inherited rather than abandoned. Git workflow is version control, the foundational reproducibility practice and the second pledge of my old reproducibility manifesto. Four of the six areas that an industry study identified as the marks of an agent-ready repository are, almost item for item, practices the open-science movement has advocated for more than a decade.

The remaining two are softer matches, but still within scope. Code style maps to readability, a value reproducibility shares but has never placed at its center: it is closer to maintainability than to replication, and the research-software community is itself still debating how much human-readability matters when a machine is the primary reader. Boundaries—the explicit list of what an agent must never touch—has the least clear analog of all, motivated as it is by the prevention of destructive automated mistakes that could not occur when only humans edited code (although human errors could also be minimized by explicitly stated boundaries). Even here the overlap is not nil: protecting raw data from being overwritten and separating sources from derived products is ordinary data-management hygiene, now made explicit because a new kind of reader can act on the files. Call it four clean matches and two partial ones. The convergence is still striking, given no one engineered it.

What does it mean that two communities with unrelated motivations arrived at nearly the same result? The temptation is to call it coincidence; it is not. Both are solving the same underlying problem: how to make a codebase usable by a reader who arrives without the author's accumulated context. Reproducibility has always served such a reader: the replicator who was not in the lab, the reviewer who never met the author, the student who inherits the code two years after its author has gone. An AI agent is an extreme instance of that reader and the most immediately present: it arrives with no shared history at the start of every session, and it arrives today rather than in two years. Practices that externalize an author's tacit knowledge into durable, legible artifacts serve all of these readers at once, because the artifact does not know whether the stranger consulting it is human or machine. In this sense reproducibility and agent-friendliness are one property and not two: each is context engineering for a reader who lacks the author's context, and they differ only in which reader is in view.

A third community has now reached the same point from a different direction, and its motivation is neither throughput nor open-science advocacy but caution. A recent vision statement from the research-software-engineering (RSE) community, prompted by concerns about the widening gap between a researcher's intent and an agent's implementation, argues that the artifacts we expect for reproducibility must expand to capture not only code and data but the reasoning and the decisions that shaped them. [10] Arriving from worry rather than enthusiasm, it reaches the conclusion the productivity study reached from the opposite mood: the durable record of how and why a system was built is what matters, and it has to be written down. When a movement built on advocacy, a study built on throughput, and a community animated by caution independently point to the same small set of artifacts, those artifacts are no longer one professor's preference or one company's house style. They are closer to a stable equilibrium: the configuration a research codebase settles into once it must answer to readers who do not share its author's mind.

4 An Optimistic Realization on AI and Science Code

The practices I have been advocating for years, recognized by many as the right ones, failed to spread because the per-session cost was high and the long-horizon return was diffuse. Now both sides of that equation have changed, and they have done so in opposite directions.

On one hand, the per-session cost has collapsed. Writing the first test used to mean learning a framework, inventing test cases, and committing to maintain the suite for the life of the project. Now you can ask an agent to write tests for an existing function and a few seconds later you have a starting point you can edit. Writing disciplined commits used to mean typing thoughtful prose every few minutes while in the middle of technical work. Now the agent can read your diff and propose a message that you accept, refine, or reject. Setting up a clean repository structure used to be a half-day of fiddling. Now a single prompt can generate the scaffolding to get you started. The friction that historically made these practices justifiable to skip is, for a researcher who has adopted agentic tools at all, now gone.

On the other hand, the long-horizon return has acquired a near-term complement. The same artifacts that pay off over five years for a future graduate student now pay off immediately for an agent. Every test you add is a permanent tripwire that protects your code from regressions, *and* it is a feedback signal the agent uses to check its own work. Every commit message that explains the why is a note for the student who inherits your codebase, *and* it is the context the agent reconstructs when it runs `git log` at the start of a new session. Every line in a project conventions file is a piece of onboarding documentation, *and* it is the prompt that prevents an agent from making the wrong assumption and going off-track.

The argument for reproducibility used to be: pay a cost today to benefit your future self, your peers, and your discipline. This is true and noble and also, demonstrably, was insufficient to motivate behavior change at scale. A new argument today is: pay almost no cost, because an agent will draft the artifact for you, and benefit *both* your future self *and* your present-day productivity, because the same agent uses the artifact every time you work. It is hard to overstate how much easier this argument is to make.

The practices themselves have not changed, nor have the reasons why they are good. The kinds of failures they prevent—results that cannot be reproduced, codebases that cannot be modified, scientific claims that cannot be defended six months after publication—are the same failures the open-science movement has been describing for years. What is new is that the bargain on offer to a working researcher has shifted from "do something hard and wait years for the payoff" to "let an agent help you make something easy and notice the payoff right away." If the reproducibility movement has been waiting for a tailwind, this could be one.

5 The Limits of the Optimistic Case

An argument that only accumulates reasons for optimism should be distrusted, and the readers I most want to persuade are the ones already inclined to distrust it. The vision statement from the Edinburgh RSE workshop is the considered, collective worry of a community that has spent years on exactly these questions, and it would be a poor tribute to wave its concerns away. Several of them are real, and the case I have made does not dissolve them. It is thus worth clarifying what this argument does and does not claim.

We begin with the sharpest objection, aimed at the heart of the argument. If an agent drafts

the test, the commit message, and the decision record, what stops a researcher from accepting all three without understanding any of them? The same tools that let a careful person externalize their context let a careless one outsource their judgment, and at scale the second pattern threatens the apprenticeship by which researchers once learned to verify and maintain code at all. This is the most serious risk in the entire picture, and I will not claim any artifacts prevent it. What I will claim is narrower and, I think, defensible: the artifacts are verification instruments, and verification is the one task in this workflow that cannot be delegated to an agent. A test you do not read is worthless; a decision record you did not reason through records nothing you know. The practice I am describing keeps the human in the position of owner and verifier precisely because the artifacts only pay off when someone exercises judgment over them. They do not guarantee understanding. They make understanding the thing you are responsible for, rather than the mechanics. That is a better division of labor than the one we had, but only for a researcher who keeps their side of it. The tool cannot supply the discipline; it can only lower the cost of exercising it.

A second concern cuts at the word reproducibility itself. AI generation is probabilistic; models change underneath you; the same prompt on the same code may produce different output next month. None of the artifacts in the catalog makes generation deterministic, clearly. But this misreads the layer at which the argument operates. The claim is not that the path by which code came to exist is reproducible; it is that the code, once it exists, is legible, tested, and accounted for. A characterization test does not care whether the function it pins was written by a human, by an agent, or by an agent on a model since deprecated; it asks only whether the behavior still holds. The artifacts do not tame the stochasticity of the generation step. What they do is convert it into something you can detect: when a non-deterministic process produces a regression, a test suite is what tells you. The thesis addresses the durable record, not the probabilistic act that produced it, and the durable record is the part that science has always asked to inspect.

A third worry is that productivity is the wrong thing to celebrate in a research culture that already rewards volume over rigor, and that making good practices cheap will only let people produce mediocre work faster. The distinction I have drawn—that the collapse in friction lowers the cost of quality rather than the cost of quantity—is real, but it is not a fix for incentives. Removing the excuse not to write a test does not make a hiring committee value the test. An institution that counts papers will go on counting papers whether or not the underlying code is sound. I claim only that the argument removes one specific obstacle that stood in the way of researchers who already wanted to do better and were defeated by the cost. Realigning what the research enterprise rewards is a larger project, and the collective statements calling for it are right to press it; this essay is not that project and should not be mistaken for it.

Some important concerns remain untouched by the argument. The provenance of the data these models were trained on, the attribution owed to the authors of that data, the widening gap between researchers who can afford frontier tools and those who cannot, and the temptation for institutions to read "AI assistance" as license to disinvest in people: these are serious, and none of them is answered by observing that a decision record is now cheap to write. They belong to a wider reckoning that the research-software community is right to demand, and they are not resolved by the narrow mechanism in scope for this paper. I raise them here so that the optimism of the preceding sections is read as bounded, not blind.

What survives is smaller than uncritical enthusiasm and also sturdier. For an individual researcher deciding how to work this week, none of the conceded limits is a reason to wait. The skill-erosion risk is a reason to keep ownership, not to forgo the artifacts; the non-determinism is a reason to lean harder on tests, not to abandon them; the incentive problem is a reason to advocate

for better evaluation, not to write worse code in the meantime. My argument is definitely not that AI agents make research reproducible on their own. It is that the practices which make research reproducible are less costly, and have acquired a second payoff that arrives instantly. That claim is narrow enough to defend and enough to act on.

6 Agents on the Road to Reproducibility

If you accept the idea that agent-readiness and reproducibility work together, then a corollary follows. Whether your project gets real value from agentic AI is inseparable from whether your code is in a state that anyone—human or agent—can work with comfortably. A messy codebase will spawn messy agentic sessions. Many researchers who report that AI tools are not living up to the promise may be seeing the consequences of a codebase that resisted structure for years. The practices that would have made their code defensible to a reviewer are the same practices that would make it now tractable to an agent.

When I pledged publicly the list of things I would do for reproducibility, in my research group, I did so with an optimism that wore down over the years. Not because the friction of keeping to it, year after year, against the pressure of deadlines and the indifference of incentives, was hard. It simply became tiresome to walk that road less travelled, with so few others. Working reproducibly remained stubbornly unrewarded. The era now beginning lowers the labor cost of honoring those pledges, while multiplying the returns. I would today add a few items the 2012 Reproducibility PI Manifesto could not have imagined: that every repository we are responsible for carries an instruction file we keep current, so that no collaborator and no agent begins in ignorance; that every function whose correctness carries a scientific claim is pinned by a test we have read and trust; that every consequential decision is recorded with the reasoning that produced it, not merely the choice that resulted; that the history of each project explains why and not only what; and that we learn to drive these tools so the record of the work writes itself, under our control, as the work proceeds. We have a whole new set of norms to follow and practices to teach our students.

One pledge underwrites all the others, one that the growing concerns in our community demand: that we remain the owners and the verifiers of everything an agent drafts in our name. The artifacts are cheap now; the judgment is not, and the judgment is the part that was always ours to grow and defend. An agent can write the test, but only we can decide if it tests the right thing. An agent can record the decision, but only we can vouch that the reasoning is sound. That responsibility does not lighten when we delegate to agents; it actually grows, because there is more output to stand behind and less labor to remind us that we are the ones standing behind it.

For most of my career I have argued that researchers should adopt the practices of professional software engineering because doing so makes our science more reliable. I now make the case more readily, for a reason I would not have predicted: the most convincing argument for working reproducibly can be made on the basis of the day's work rather than the distant future. The test you once wrote out of duty, and skipped under deadline, you now write because an agent drafts it in seconds and uses it to check its own work an hour later. The payoff I used to ask people to take on faith can now be shown to be immediate. That, not any argument I could have made before, is what may change how we work from now on.

References

- [1] Lorena A. Barba. The hard road to reproducibility. *Science*, 354(6308):142, 2016. doi: 10.1126/science.354.6308.142.
- [2] Lorena A. Barba. Reproducibility PI manifesto. figshare, 2012. Presentation, version 1.
- [3] Generative AI for engineering research. Course website, n.d. URL <https://barbagroup.github.io/mae6291-genai/>.
- [4] Matt Nigh. How to write a great agents.md: Lessons from over 2,500 repositories. The GitHub Blog, November 2025. URL <https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/>.
- [5] Michael Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004. ISBN 0131177052.
- [6] Roscoe A. Bartlett. Working effectively with legacy code. Better Scientific Software, June 2019. URL <https://bssw.io/items/working-effectively-with-legacy-code>. Book review.
- [7] Conventional commits, n.d. URL <https://www.conventionalcommits.org/>. Specification, version 1.0.0.
- [8] Birgitta Böckeler. Context engineering for coding agents, February 2026. URL <https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html>.
- [9] Architectural decision records (ADRs). Community-maintained documentation, n.d. URL <https://adr.github.io/>.
- [10] Daniel S. Katz et al. Research software in an age of AI-assisted development: Reflections from edinburgh. Zenodo, May 2026. 27 May 2026.